Particelle Cariche in Campo Elettromagnetico

Stefania Decesari

20 Maggio 2025

1 Introduzione

Lo scopo di questo progetto consiste nello studiare il moto di particelle cariche non relativistiche immerse in un campo elettromagnetico costante, in alcune semplici configurazioni di campo. Il primo caso trattato comprende lo studio del moto di una particella immersa in un campo magnetico costante e un campo elettrico nullo; nel secondo caso si considera un campo elettrico costante parallelo a un campo magnetico costante; nel terzo caso il campo magnetico e il campo elettrico sono presi perpendicolari tra loro. Nell'ultimo caso vengono, invece, studiate più particelle in una configurazione di campo magnetico a "X point". I due algoritmi numerici utilizzati per la risoluzione dell'equazione differenziale che caratterizza questo problema fisico sono: il metodo di Runge-Kutta del quarto ordine e l'algoritmo di Boris del secondo ordine. E' utile ricorrere all'integrazione numerica per questo tipo di problema poiché la soluzione analitica dell'equazione del moto è ottenibile solo in semplici configurazioni del sistema fisico in esame.

2 Teoria

Il moto di una particella carica non relativistica in presenza di campo elettromagnetico è descritto dalla forza di Lorentz:

$$\mathbf{F}_q = q(\mathbf{E} + \frac{\boldsymbol{v}}{c} \wedge \mathbf{B}) \tag{1}$$

Quindi l'equazione del moto può essere scritta nel sistema cgs come:

$$\frac{d\boldsymbol{v}}{dt} = \frac{q}{mc}(c\mathbf{E} + \boldsymbol{v} \wedge \mathbf{B}) \tag{2}$$

in cui q è la carica della particella, $\boldsymbol{v}$ la sua velocità tridimensionale e m la sua massa, c è la velocità della luce e $\mathbf{E}$ e $\mathbf{B}$ sono rispettivamente il vettore campo elettrico e il vettore campo magnetico. Si può scomporre l'equazione del moto nelle tre componenti spaziali:

$$\begin{aligned} \frac{dv_x}{dt} &= \alpha(cE_x + v_yB_z - v_zB_y) \\ \frac{dv_y}{dt} &= \alpha(cE_y + v_zB_x - v_xB_z) \\ \frac{dv_z}{dt} &= \alpha(cE_z + v_xB_y - v_yB_x) \end{aligned} \tag{3}$$

Da queste tre equazioni è evidente che il campo magnetico contribuisca solo per la variazione della direzione della velocità, non del modulo di essa, per via del prodotto vettoriale.

Per quanto riguarda le unità di misura nel sistema cgs, se si suppone di considerare una particella con carica uguale alla carica elementare di un elettrone, e $= -4.80 \times 10^{-10}$ statC, massa pari alla massa dell'elettrone, $m_e = 9.11 \times 10^{-28}$ g, sapendo che la velocità della luce nel vuoto è c $= 3 \times 10^{10} \frac{\text{cm}}{\text{s}}$, si ottiene che il fattore moltiplicativo davanti alle parentesi tonde è: $\alpha = \frac{e}{m_e c} = 1.76 \times 10^7 \text{cm}^{\frac{1}{2}} \text{g}^{-\frac{1}{2}} \text{s}^{-1}$.

Si è posta questa costante α pari a 1. Allora per ottenere stime di osservabili fisiche dai risultati della simulazione, si dovrà considerare la scala temporale in 10^{-7}s. Infatti se si denotano le quantità fisiche come ad esempio il tempo $t = t_{cgs} t_c$, dove t_{cgs} è una costante in unità cgs, la quale rappresenta l'unità di scala scelta, mentre t_c è la quantità adimensionale che si utilizza nel codice, l'equazione del moto diventa:

$$\frac{l_{cgs}}{t_{cgs}^2}\frac{d\boldsymbol{v}_c}{dt_c} = \frac{e}{m_e c}\left(\frac{m_e c}{e}\frac{l_{cgs}}{t_{cgs}^2}\boldsymbol{E}_c + \frac{l_{cgs}}{t_{cgs}}\boldsymbol{v}_c \wedge \frac{m_e}{e}\frac{1}{t_{cgs}}\boldsymbol{B}_c\right) \tag{4}$$

mentre l'equazione adimensionale che si risolve nel codice è:

$$\frac{d\boldsymbol{v}_c}{dt_c} = (\mathbf{E_c} + \boldsymbol{v}_c \wedge \mathbf{B_c}) \tag{5}$$

in cui si è posta la costante moltiplicativa pari a 1, quindi questa simulazione non descrive alcun sistema fisico, ma risulta uno scaling del problema posto e lo scaling è rappresentato proprio da questa costante. Allora per trasformare i tempi utilizzati nella simulazione in tempi fisici in secondi si dovrà applicare la seguente trasformazione:

$$t = t_{cgs}t_c = \frac{m_e c}{e}t_c \ \text{ s} = \frac{9,11\times 10^{-28}\ 3\times 10^{10}}{4,80\times 10^{-10}}t_c \ \text{ s} \approx 5,69\times 10^{-8}t_c \ \text{ s} \tag{6}$$

Inoltre ogni volta che nella simulazione viene indicato **E** si sottintende **cE**.

3 Metodi Numerici Utilizzati

Il metodo di Runge-Kutta del quarto ordine è un metodo iterativo a singolo passo, il quale si basa sulle espansioni in serie di Taylor e sull'approssimazione dell'integrale con la regola di Simpson. Richiede quattro valutazioni della funzione right-hand side R per ciascun passo Δt, in diversi punti dell'intervallo della discretizzazione:

$$\begin{aligned}
k_1 &= R(t_n, Y_n) \\
k_2 &= R\left(t_n + \frac{\Delta t}{2},\ Y_n + \frac{\Delta t}{2}k_1\right) \\
k_3 &= R\left(t_n + \frac{\Delta t}{2},\ Y_n + \frac{\Delta t}{2}k_2\right) \\
k_4 &= R\left(t_n + \Delta t,\ Y_n + \Delta t\, k_3\right) \\
Y_{n+1} &= Y_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned} \tag{7}$$

Quindi vengono calcolati quattro incrementi diversi: il primo viene calcolato valutando la funzione R nel punto iniziale dell'intervallo della discretizzazione temporale, il secondo e il terzo nel punto al centro e l'ultimo nel punto finale. La funzione alla fine dello step (n+1)-esimo è data da quella n-esima, incrementata di Δt volte la media pesata di 4 diverse valutazioni della derivata.

L'algoritmo di Boris è invece un integratore simplettico, un'estensione dell'algoritmo Position-Verlet che lo rende adatto per studiare particelle in campo magnetico, infatti il metodo di Boris è in grado di gestire casi in cui la forza dipende dalla velocità come accade per la forza di Lorentz.

$$\begin{aligned}
x_{n+\frac{1}{2}} &= x_n + \frac{\Delta t}{2}v_n \\
v_- &= v_n + \frac{\Delta t}{2}E_{n+\frac{1}{2}} \\
v_+ &= v_- + 2\frac{v_- + v_- \wedge \frac{\Delta t}{2}B_n}{1 + \frac{\Delta t^2}{4}B_n^2} \\
v_{n+1} &= v_+ + \frac{\Delta t}{2}E_{n+\frac{1}{2}} \\
x_{n+1} &= x_{n+\frac{1}{2}} + \frac{\Delta t}{2}v_{n+1}
\end{aligned} \tag{8}$$

Viene prima aggiornata la posizione di mezzo passo Δt, viene poi aggiornata la velocità di mezzo passo Δt con la sola azione del campo elettrico, in seguito con il solo campo magnetico (rotazione), e infine con il campo elettrico di mezzo passo Δt. Viene poi riaggiornata la posizione.

4 Risultati

4.1 Semplice Rotazione

In questo primo caso viene analizzata la situazione in cui il vettore campo elettrico e il vettore campo magnetico sono: $\mathbf{E} = (0.0, 0.0, 0.0)$, $\mathbf{B} = (0.0, 0.0, 1.0)$. Ciò viene implementato nel codice con un array da sei componenti, le prime tre corrispondenti alle tre componenti del campo elettrico E_x, E_y, E_z e le ultime tre corrispondenti alle tre componenti del campo magnetico B_x, B_y, B_z. In questa configurazione di campo elettromagnetico, l'equazione del moto in componenti (3) diventa:

$$\begin{aligned} \frac{dv_x}{dt} &= v_y B_z \\ \frac{dv_y}{dt} &= -v_x B_z \\ \frac{dv_z}{dt} &= 0 \end{aligned} \tag{9}$$

avendo posto la costante $\alpha = 1$. Come posizioni e velocità iniziali della particella si sono prese $\mathbf{x} = (0.0, 1.0, 0.0)$ e $\boldsymbol{v} = (1.0, 0.0, 0.0)$. La soluzione dell'equazione del moto è la curva nel piano: (x = `sin`(t), y = `cos`(t)), dove `t` è un parametro utilizzato come tempo della simulazione. La particella secondo la teoria dovrebbe quindi compiere una semplice girazione sul piano xy in senso orario.
Si è dapprima verificata la validità dei due algoritmi utilizzati, confrontando la soluzione numerica con quella analitica al variare del passo Δt.

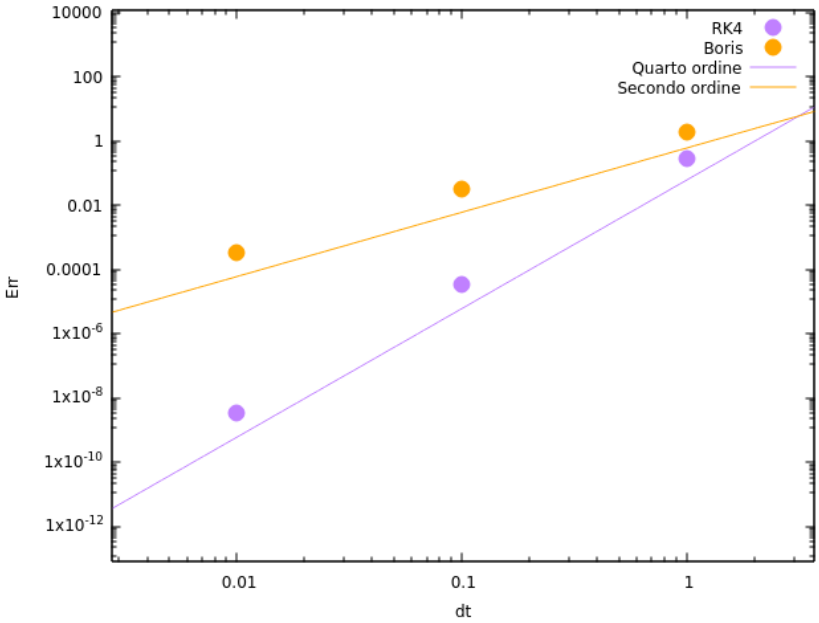

Figure 1: Test di Convergenza dei due algoritmi utilizzati, a `t` = 100 .

Dalla figura (1) si evince che l'algoritmo Runge-Kutta 4 mostra una convergenza del quarto ordine, mentre l'algoritmo di Boris mostra una convergenza del secondo ordine. Pertanto i due algoritmi sono validi e restituiscono la precisione che si richiede.

Tuttavia, nonostante il quarto ordine del metodo di Runge-Kutta esso non preserva le caratteristiche geometriche del problema come invece fa l'algoritmo di Boris. Infatti se si grafica l'energia della particella in funzione del tempo a diversi timesteps:

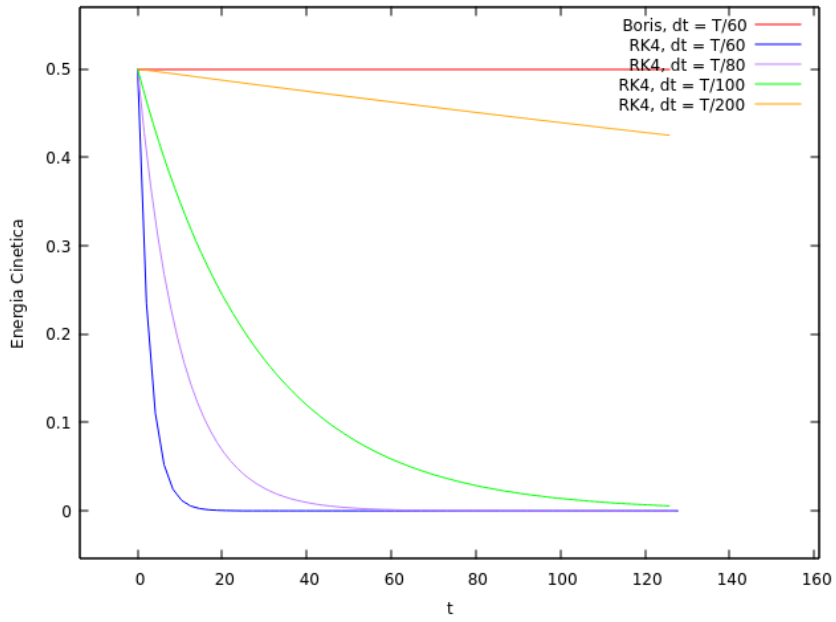

Figure 2: Energia cinetica della particella per un tempo `t` tale da farle compiere 20 girazioni.

è evidente che l'algoritmo di Runge-Kutta 4 non preservi l'energia per timesteps elevati, al contrario dell'algoritmo di Boris.
Si è poi confrontata la soluzione numerica nel piano xy con la soluzione esatta a due diversi valori di Δt, a `t` $= 100$:

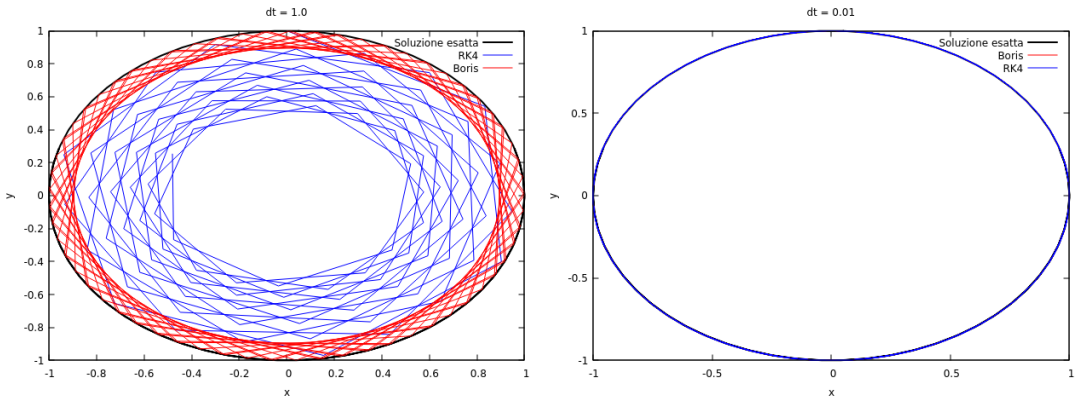

Figure 3: Sinistra: Soluzioni numeriche e soluzione esatta a confronto per un $\Delta t = 1.0$. Destra: Soluzioni numeriche e soluzione esatta a confronto per un $\Delta t = 0.01$ (tutte e tre coincidono).

Si osserva che, con un passo temporale pari a 1.0, l'algoritmo di Runge-Kutta presenta difficoltà di convergenza, mentre riducendo opportunamente il passo temporale si ottiene una convergenza accurata della soluzione. E' anche evidente che l'algoritmo di Boris per un Δt pari a 1.0 è comunque più efficiente del metodo di Runge-Kutta 4 nonostante il minore ordine di convergenza, questo perché l'algoritmo di Boris è più adatto per problemi fisici in cui la forza dipende dalla velocità, come nel caso della forza di Lorentz. Inoltre il metodo di Runge-Kutta non preserva le proprietà geometriche del sistema, al contrario dell'algoritmo di Boris, come già evidenziato. Di seguito viene riportata la traiettoria della particella in

tre dimensioni per un tempo `t` tale da farle compiere 20 girazioni.

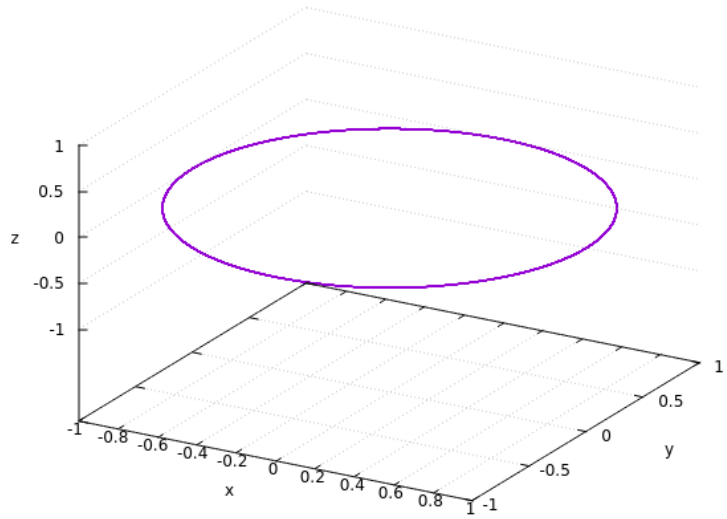

Figure 4: Traiettoria della particella in 3 dimensioni utilizzando l'algoritmo di Boris, con $\Delta t = 0.01$.

4.2 Campo Elettrico e Campo Magnetico paralleli

Nel secondo caso si è studiato il moto di una particella carica in una configurazione di campo elettromagnetico come: $\mathbf{E} = (0.0, 0.0, 0.6)$, $\mathbf{B} = (0.0, 0.0, 1.0)$. Si è fatta attenzione a rispettare la condizione $|\mathbf{E}| < |\mathbf{B}|$ in modo da non incorrere in casi relativistici che esulano dalla trattazione di questo progetto. Come condizioni iniziali della particella si sono considerate come nel primo caso $\mathbf{x} = (0.0, 1.0, 0.0)$ e $\boldsymbol{v} = (1.0, 0.0, 0.0)$. Si tratta della situazione analoga al caso precedente ma con l'aggiunta di un moto accelerato lungo l'asse `z`. L'equazione del moto in componenti pertanto diventa:

$$\begin{aligned} \frac{dv_x}{dt} &= v_y B_z \\ \frac{dv_y}{dt} &= -v_x B_z \\ \frac{dv_z}{dt} &= E_z \end{aligned} \tag{10}$$

avendo sempre posto la costante α pari a 1 . La particella percorrerà quindi una traiettoria a elica con passo variabile nel tempo, poiché compierà sempre una rotazione sul piano `xy` (x = `sin`(t), y = `cos`(t)) ma essa sarà anche accelerata lungo l'asse `z` con accelerazione $a_z = 0.6$. Segue il grafico che mostra la traiettoria della particella simulata con l'algoritmo di Boris in tre dimensioni.

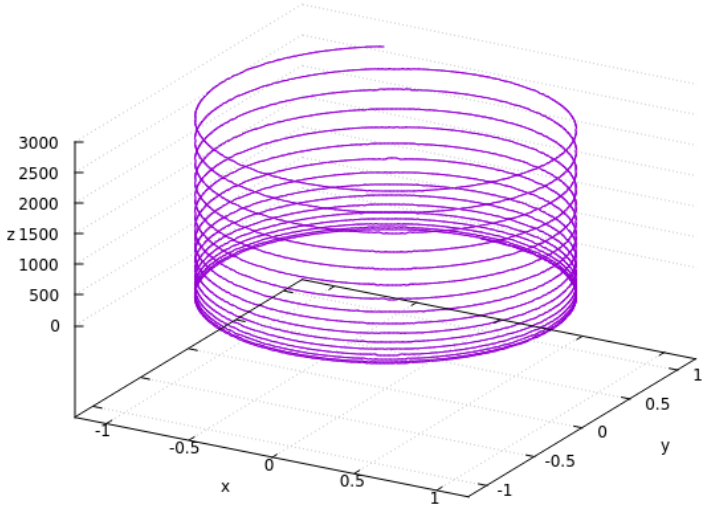

Figure 5: Traiettoria della particella in 3 dimensioni utilizzando l'algoritmo di Boris, con $\mathtt{t} = 100.0$ e $\Delta t = 0.01$.

4.3 Campo Elettrico e Campo Magnetico perpendicolari

Il terzo caso studia il moto di una particella carica in una configurazione di campo elettromagnetico come la seguente: $\mathbf{E} = (0.0, 0.6, 0.0)$, $\mathbf{B} = (0.0, 0.0, 1.0)$. Come posizioni e velocità iniziali della particella si sono prese nuovamente $\mathbf{x} = (0.0, 1.0, 0.0)$ e $\boldsymbol{v} = (1.0, 0.0, 0.0)$. Le equazioni del moto diventano:

$$\begin{aligned} \frac{dv_x}{dt} &= v_y B_z \\ \frac{dv_y}{dt} &= E_y - v_x B_z \\ \frac{dv_z}{dt} &= 0 \end{aligned} \tag{11}$$

avendo sempre posto la costante α pari a 1. Si nota subito che il moto sarà un moto planare sul piano `xy` poiché la terza equazione implica che v_z sia costante e come valore iniziale essa assume zero. La traiettoria è, più precisamente, una cicloide, infatti le equazioni (11) mostrano che, al primo istante di tempo, date le condizioni iniziali della particella, essa abbia velocità lungo l'asse `y` pari a zero, pertanto la prima equazione si annulla e rimane solo una accelerazione lungo l'asse `y`, a cui contribuiscono la componente `y` del campo elettrico e la componente `z` del campo magnetico. All'inizio prevarrà il contributo negativo che farà muovere la particella nel verso negativo dell'asse `y`. Nel frattempo quindi v_y assumerà un valore diverso da zero che non renderà più nulla la prima equazione, si avrà allora anche una accelerazione lungo l'asse `x` che porterà a variare la velocità lungo l'asse `x`. Nel momento in cui la velocità lungo l'asse `x` diminuirà in modo da non far prevalere più il contributo negativo del campo magnetico sul contributo positivo del campo elettrico nella seconda equazione, la particella invertirà il suo moto lungo l'asse `y`, in modo da creare una oscillazione. Si noti che è presente un moto complessivo di deriva lungo l'asse `x` poiché v_x rimane complessivamente positiva. Si è in seguito, anche in questo caso, voluto mostrare che i due algoritmi ad alti timesteps non riproducano in modo accurato la soluzione esatta, mentre a timesteps adeguatamente piccoli essi siano efficienti.

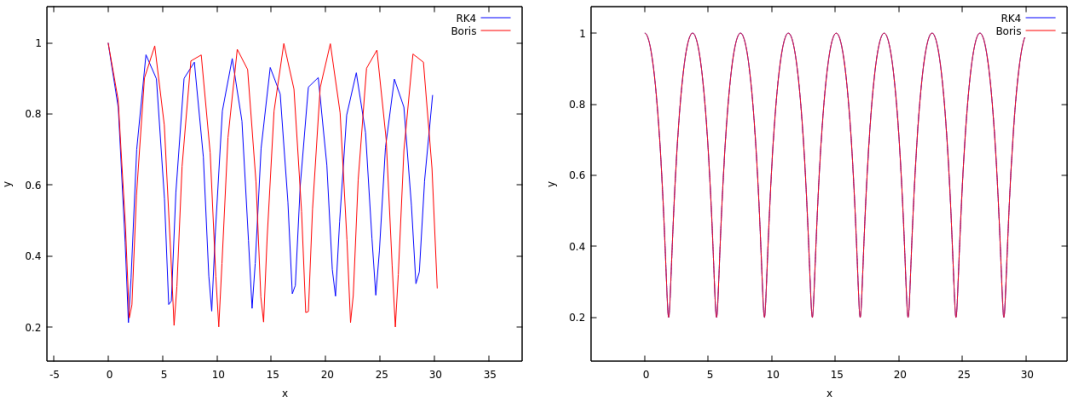

Figure 6: Sinistra: Soluzioni numeriche a confronto con $\Delta t = 1.0$. Destra: Soluzioni numeriche a confronto con $\Delta t = 0.01$ (tutte e due coincidono e riproducono bene la soluzione teorica).

Si nota nuovamente che l'algoritmo di Runge-Kutta non preservi la geometria del sistema rispetto all'algoritmo di Boris come si evince dal fatto che le oscillazioni tendano a collassare.
Si mostra infine la traiettoria della particella in tre dimensioni con l'utilizzo dell'algoritmo di Boris con $\Delta t = 0.01$.

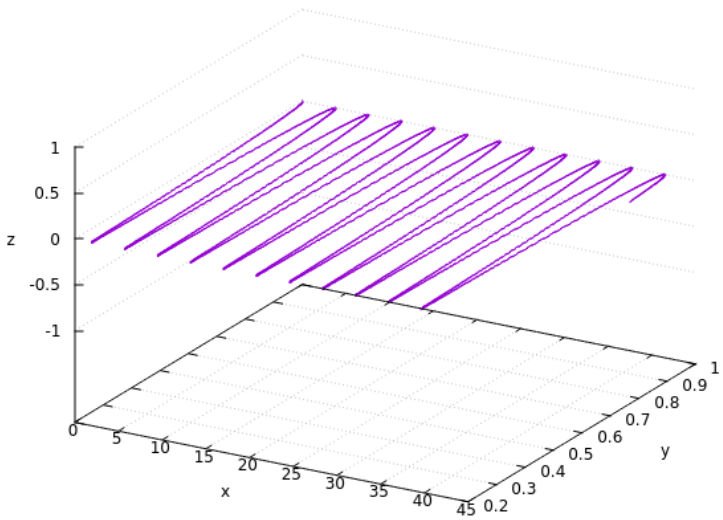

Figure 7: Traiettoria della particella in 3 dimensioni utilizzando l'algoritmo di Boris, con $\mathtt{t} = 100.0$ e $\Delta t = 0.01$.

4.4 Configurazione a X point

Si tratta di un caso semplificato, ma importante per comprendere alcuni fenomeni in Fisica del Plasma, ad esempio il fenomeno di Riconnessione Magnetica, in cui avviene una ricostruzione topologica del campo magnetico, causata da un cambiamento nella connessione delle linee di campo.
In questo caso viene analizzata la configurazione di campo elettromagnetico: $\mathbf{E} = (0.0, 0.0, 0.5)$, $\mathbf{B} = (\mathtt{y/1000}, \mathtt{x/1000}, 0.0)$, quindi le linee di campo magnetico sono delle iperboli che riproducono un "X point" nel piano `xy`, mentre il campo elettrico è costante lungo l'asse `z`. In questo ultimo caso non si è più considerata una sola particella, ma 3000 (nella simulazione si è posto il parametro `NPART = 3000`) . Si sono poi posizionate le particelle randomicamente in un dominio quadrato di `[-1000, 1000] x [-1000,`

1000] e si sono inizializzate le velocità di queste ultime in modo che il modulo fosse pari a 0.1, mentre la direzione potesse variare randomicamente (si sono utilizzati degli angoli randomici). Le equazioni del moto, pertanto diventano:

$$\begin{aligned} \frac{dv_x}{dt} &= -v_z B_y \\ \frac{dv_y}{dt} &= v_z B_x \\ \frac{dv_z}{dt} &= E_z \end{aligned} \tag{12}$$

Si è sempre mantenuta la costante α pari a 1.
Si è allora eseguita la simulazione. Si riportano i grafici delle posizioni x e y delle particelle e della loro energia a diversi istanti di tempo:

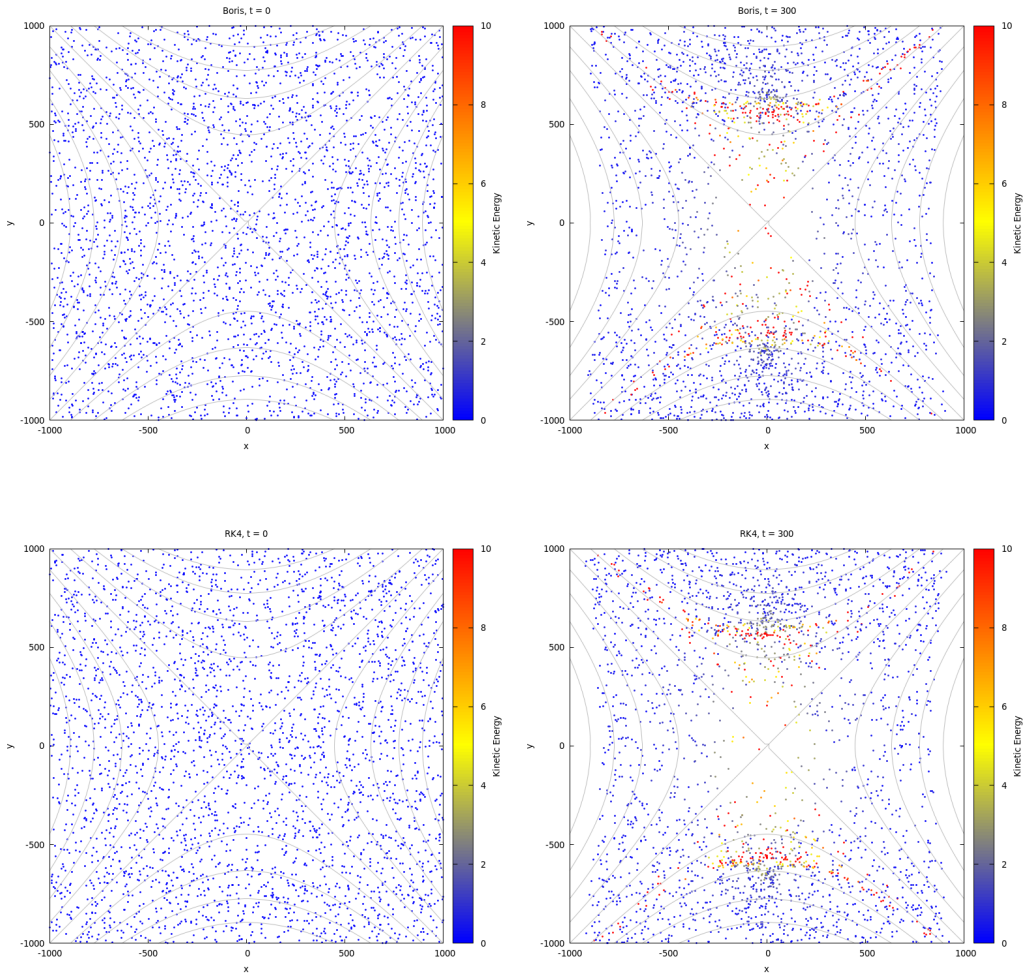

Figure 8: In alto si riporta il risultato ottenuto con l'algoritmo di Boris a diversi istanti di tempo: a sinistra all'istante t = 0, mentre a destra all'istante t = 300. In basso si riporta il risultato ottenuto con l'algoritmo di Runge-Kutta 4 a diversi istanti di tempo: a sinistra all'istante t = 0, mentre a destra all'istante t = 300. Per entrambi i metodi numerici si è mantenuto un Δt pari a 0.01 .

Nella figura (8) viene mostrata l'azione accelerante del campo, infatti l'energia cinetica delle particelle aumenta con il passare del tempo e esse tendono a disporsi in una configurazione "a mezzaluna". Nell'intorno del punto x = 0, y = 0 il campo magnetico è debole e le particelle tendono presto ad abbandonarlo. Si noti che vengono visualizzate solo le particelle all'interno del dominio, quindi a istanti

successivi alcune particelle avranno già abbandonato il dominio.
Per verificare che il meccanismo di accelerazione sia dovuto all'azione del campo in configurazione a "X point" si è graficato un istogramma in cui si riporta la distribuzione energetica delle particelle a `t` = 0 a confronto con quella a `t` = 900.

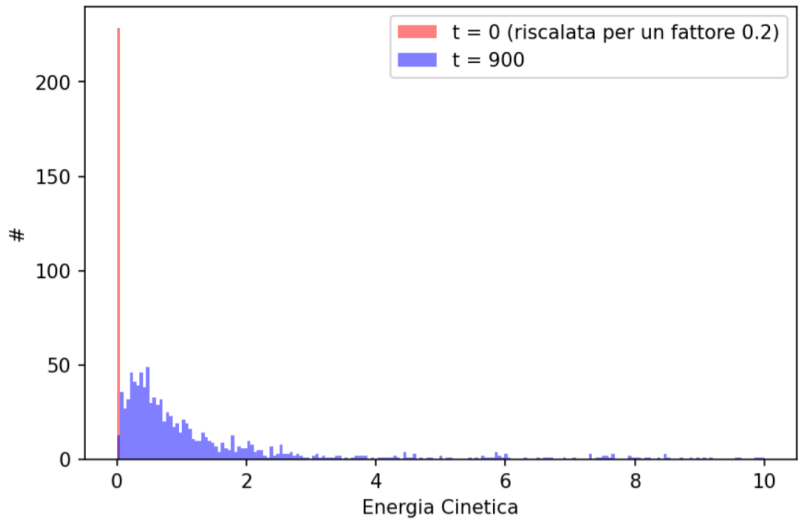

Figure 9: Distribuzione di energia delle particelle a `t` = 0 e a `t` = 900 .

Si è riscalata la distribuzione a `t` = 0 per poter visualizzare meglio la distribuzione a `t` = 900. All'inizio le particelle hanno il medesimo valore di energia poiché si è inizializzato il modulo della velocità delle particelle in modo che fosse 0.1.
Inoltre si è anche graficata la distribuzione di energia di 10000 particelle (`NPAR` = 10000) inizializzate con modulo della velocità a 0.1.

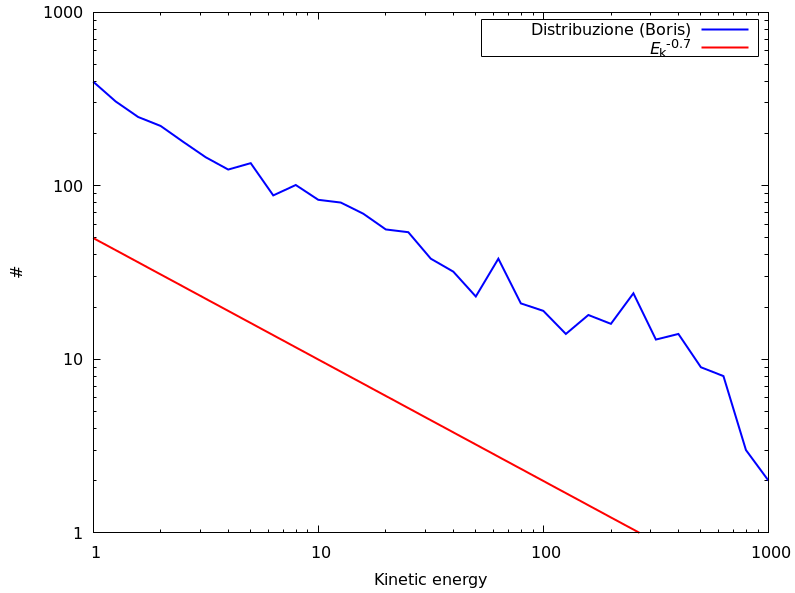

Figure 10: Distribuzione di energia delle particelle confrontata con una legge di potenze .

E' evidente che la distribuzione di energia segua una legge di potenza, pertanto si conclude che l'azione accelerante che subiscono le particelle è dovuta alla presenza di campo elettromagnetico nella configurazione analizzata.

5 Conclusioni

In questo progetto, per mezzo della scrittura di una simulazione, si è analizzata l'azione della forza di Lorentz su una particella carica in tre diverse configurazioni di campo elettromagnetico. Si è, poi, analizzato un caso in cui le particelle erano più di una, in una configurazione di campo a "X point". Nel primo caso si è eseguito uno studio di convergenza per verificare l'accuratezza dei due metodi numerici utilizzati: l'algoritmo di Runge-Kutta del quarto ordine e l'algoritmo di Boris del secondo ordine. In seguito si è confrontata la soluzione numerica con la soluzione esatta e in tutti e tre i primi casi si è visualizzata la traiettoria della particella in tre dimensioni in accordo con la teoria. Nell'ultimo caso si è visualizzata la posizione delle particelle sul piano `xy` a istanti successivi di tempo e si è studiata infine la loro accelerazione.

6 Inserting Code

```
#include<iostream>
#include<cmath>
#include<ctime>
#include<iomanip>
#include<fstream>
#include<cstdlib>

using namespace std;

#define STAGE 1       //Da 1 a 4
#define TL 100.0      //Tempo finale della simulazione
#define NPAR 3000     //Numero di particelle per il caso 4
#define XL 1000.0     //Range dell'asse x per il caso 4
#define YL 1000.0     //Range dell'asse y per il caso 4
#define NMAX_EQ 16

void RK4Step(double t, double *Y, double *EB,
             void (*Ydot)(double, double *, double *, double *),
             double dt, int Neq);
void BorisStep(double t, double *Y, double *EB, double dt);
void dYdt(double t, double *Y, double *EB, double *R);
void EBFields(double *Y, double *EB);
void InitialConditions(double *Y);

int main()
{
  int i, j;
  double EB[6];
  int Neq = 6;
  double Yrk4[Neq], Ybor[Neq];
  double t = 0.0;
  double dt = 0.01;
  int count = 0;
  double errRk4 = 0.0, errBor = 0.0;
  double dt_val[] = {1.0, 0.1, 0.01};

  ofstream fdata_RK4, fdata_Bor, fdata_Err;

  fdata_RK4.open("RK4.dat");
  fdata_Bor.open("Bor.dat");
  fdata_Err.open("Err.dat");
  fdata_RK4 << setiosflags(ios::scientific);
  fdata_Bor << setiosflags(ios::scientific);
  fdata_Err << setiosflags(ios::scientific);

  #if STAGE == 1 or STAGE == 2 or STAGE == 3    //Casi di singola particella

  InitialConditions(Yrk4);
  InitialConditions(Ybor);

  fdata_RK4 << t << " " << Yrk4[0] << " " << Yrk4[1] << " " << Yrk4[2] << " "
            << 0.5*(Yrk4[3]*Yrk4[3]+Yrk4[4]*Yrk4[4]+Yrk4[5]*Yrk4[5])   << endl;
  fdata_Bor << t << " " << Ybor[0] << " " << Ybor[1] << " " << Ybor[2] << " "
            << 0.5*(Ybor[3]*Ybor[3]+Ybor[4]*Ybor[4]+Ybor[5]*Ybor[5])   << endl;

  for(j = 0; j < 3; j++){
     dt = dt_val[j];
     InitialConditions(Yrk4);
     InitialConditions(Ybor);
     t = 0.0;

     for(i = 0; t < TL; i++){
        RK4Step(t, Yrk4, EB, dYdt, dt, Neq);
        BorisStep(t, Ybor, EB, dt);
        t += dt;
        fdata_RK4 << t << " " << Yrk4[0] << " " << Yrk4[1] << " " << Yrk4[2] << " "
                  << 0.5*(Yrk4[3]*Yrk4[3]+Yrk4[4]*Yrk4[4]+Yrk4[5]*Yrk4[5])   << endl;
```

```
            fdata_Bor << t << " " << Ybor[0] << " " << Ybor[1] << " " << Ybor[2] << " "
                      << 0.5*(Ybor[3]*Ybor[3]+Ybor[4]*Ybor[4]+Ybor[5]*Ybor[5])   << endl;
        }
        fdata_RK4 << endl << endl;
        fdata_Bor << endl << endl;

        errRk4 = sqrt((Yrk4[0] - sin(t))*(Yrk4[0] - sin(t)) +
        (Yrk4[1] - cos(t))*(Yrk4[1] -   cos(t)));

        errBor = sqrt((Ybor[0] - sin(t))*(Ybor[0] - sin(t)) +
        (Ybor[1] - cos(t))*(Ybor[1] - cos(t)));

        fdata_Err << dt << " " << errRk4 << " " << errBor << " " << endl;
    }
    elif STAGE == 4    //Caso di molteplici particelle

    srand48(time(NULL));

    for(j = 0; j < NPAR; j++){
        InitialConditions(Yrk4);
        t = 0.0;
        for(i = 0; t < TL; i++){
            count = 0;
            if(Yrk4[0] < -XL || Yrk4[0] > XL || Yrk4[1] < -YL || Yrk4[1] > YL){
              count = 1;
              break;
            }
            RK4Step(t, Yrk4, EB, dYdt, dt, Neq);
            t += dt;
        }
        if(count == 0){
            fdata_RK4 << t << " " << Yrk4[0] << " " << Yrk4[1] << " " << Yrk4[2] << " "
                      << 0.5*(Yrk4[3]*Yrk4[3]+Yrk4[4]*Yrk4[4]+Yrk4[5]*Yrk4[5])   << endl;
        }
    }
    for(j = 0; j < NPAR; j++){
        InitialConditions(Ybor);
        t = 0.0;
        for(i = 0; t < TL; i++){
            count = 0;
            if(Ybor[0] < -XL || Ybor[0] > XL || Ybor[1] < -YL || Ybor[1] > YL){
              count = 1;
              break;
            }
            BorisStep(t, Ybor, EB, dt);
            t += dt;
        }
        if(count == 0){
            fdata_Bor << t << " " << Ybor[0] << " " << Ybor[1] << " " << Ybor[2] << " "
                      << 0.5*(Ybor[3]*Ybor[3]+Ybor[4]*Ybor[4]+Ybor[5]*Ybor[5])   << endl;
        }
    }

    #endif

    fdata_RK4.close();
    fdata_Bor.close();
    fdata_Err.close();

    return(0);
 }
////////////////////////////////////////////////////////////////////////
void InitialConditions(double *Y)
//Assegna le posizioni e le velocita' iniziali di una particella
//
////////////////////////////////////////////////////////////////////////
{
     #if STAGE == 1 or STAGE == 2 or STAGE == 3
     Y[0] = 0.0, Y[1] = 1.0, Y[2] = 0.0;
```

```
    Y[3] = 1.0, Y[4] = 0.0, Y[5] = 0.0;

    #elif STAGE == 4
    double x, y, vx, vy, theta;
    x = (drand48()*2.0*XL) - XL;
    y = (drand48()*2.0*YL) - YL;
    theta = drand48()*2.0*M_PI;
    vx = 0.1*cos(theta);
    vy = 0.1*sin(theta);

    Y[0] = x,  Y[1] = y,  Y[2] = 0.0;
    Y[3] = vx, Y[4] = vy, Y[5] = 0.0;

    #endif
}
////////////////////////////////////////////////////////////////////////
void EBFields(double *Y, double *EB)
//Calcola le 4 diverse configurazioni di campo elettromagnetico
//
////////////////////////////////////////////////////////////////////////
{
    double l = 0.001;

    #if STAGE == 1
    EB[0] = 0.0, EB[1] = 0.0, EB[2] = 0.0;
    EB[3] = 0.0, EB[4] = 0.0, EB[5] = 1.0;

    #elif STAGE == 2
    EB[0] = 0.0, EB[1] = 0.0, EB[2] = 0.6;
    EB[3] = 0.0, EB[4] = 0.0, EB[5] = 1.0;

    #elif STAGE == 3
    EB[0] = 0.0, EB[1] = 0.6, EB[2] = 0.0;
    EB[3] = 0.0, EB[4] = 0.0, EB[5] = 1.0;

    #elif STAGE == 4
    EB[0] = 0.0,    EB[1] = 0.0,    EB[2] = 0.5;
    EB[3] = l*Y[1], EB[4] = l*Y[0], EB[5] = 0.0;

    #endif


}
////////////////////////////////////////////////////////////////////////
void dYdt(double t, double *Y, double *EB, double *R)
//La funzione right-hand side
//
////////////////////////////////////////////////////////////////////////
{
    EBFields(Y, EB);

    double x  = Y[0],  y = Y[1],   z  = Y[2];
    double vx = Y[3],  vy = Y[4],  vz = Y[5];
    double Ex = EB[0], Ey = EB[1], Ez = EB[2];
    double Bx = EB[3], By = EB[4], Bz = EB[5];


    R[0] = vx, R[1] = vy, R[2] = vz;
    R[3] = Ex + vy*Bz - vz*By;       //Forza di Lorentz
    R[4] = Ey + vz*Bx - vx*Bz;
    R[5] = Ez + vx*By - vy*Bx;

}

////////////////////////////////////////////////////////////////////////
void RK4Step(double t, double *Y, double *EB,
             void (*Ydot)(double, double *, double *, double *),
             double dt, int Neq)
//Metodo di Runge-Kutta del quarto ordine
```

```
//
////////////////////////////////////////////////////////////////////////
{

    int i;
    double k1[NMAX_EQ], k2[NMAX_EQ], k3[NMAX_EQ], k4[NMAX_EQ], Y1[NMAX_EQ];

    Ydot(t, Y, EB, k1);
    for(i = 0; i < Neq; i++){

        Y1[i] = Y[i] + 0.5 * dt * k1[i];

    }

    Ydot(t + 0.5*dt, Y1, EB, k2);
    for(i = 0; i < Neq; i++){

        Y1[i] = Y[i] + 0.5 * dt * k2[i];
    }

    Ydot(t + 0.5*dt, Y1, EB, k3);
    for(i = 0; i < Neq; i++){

        Y1[i] = Y[i] +  dt * k3[i];
    }

    Ydot( t + dt, Y1, EB, k4);
    for(i = 0; i < Neq; i++){

        Y[i] += dt*(k1[i] + 2.0*(k2[i] + k3[i]) + k4[i])/6.0;
    }
}
////////////////////////////////////////////////////////////////////////
void BorisStep(double t, double *Y, double *EB, double dt)
//Algoritmo di Boris del secondo ordine
//
////////////////////////////////////////////////////////////////////////
{

    EBFields(Y, EB);
    int i;
    double h = 0.5*dt;
    double den = 1.0 + h*h*(EB[3]*EB[3]+EB[4]*EB[4]+EB[5]*EB[5]);
    double vminus[3], vplus[3];

    for (i = 0; i < 3; i++) Y[i] += h*Y[i+3];

    for (i = 0; i < 3; i++) Y[i+3] += h*EB[i];

    vminus[0] = h*(Y[4]*EB[5] - Y[5]*EB[4]);
    vminus[1] = h*(Y[5]*EB[3] - Y[3]*EB[5]);
    vminus[2] = h*(Y[3]*EB[4] - Y[4]*EB[3]);

    for (i = 0; i < 3; i++) vminus[i] += Y[i+3];

    vplus[0] = h*(vminus[1]*EB[5] - vminus[2]*EB[4]);
    vplus[1] = h*(vminus[2]*EB[3] - vminus[0]*EB[5]);
    vplus[2] = h*(vminus[0]*EB[4] - vminus[1]*EB[3]);

    for (i = 0; i < 3; i++) Y[i+3] += 2.0*vplus[i]/den;

    for (i = 0; i < 3; i++) Y[i+3] += h*EB[i];

    for (i = 0; i < 3; i++) Y[i] += h*Y[i+3];

}
```